Laboratory Activity building a Flutter app that connects to a MySQL database via a PHP API hosted on InfinityFree.

This tutorial covers setting up the backend, creating the Flutter frontend, and testing the complete application.

□ Prerequisites

- **Flutter SDK**: Ensure Flutter is installed on your system.
- **InfinityFree Account**: Sign up at https://www.infinityfree.net.
- **Code Editor**: Use an editor like VS Code or Android Studio.
- **Postman or Browser**: For testing API endpoints.

🗄 □ Backend Setup (PHP & MySQL on InfinityFree)

1. Create a MySQL Database

1. Log in to your InfinityFree control panel.
2. Navigate to **MySQL Databases**.
3. Create a new database (e.g., `epiz_12345678_flutterdb`).
4. Note the following credentials:
   - **DB Name**: `epiz_12345678_flutterdb`
   - **Username**: `epiz_12345678`
   - **Password**: Your hosting account password (not the client area password)
   - **Host**: e.g., `sql311.epizy.com`Stack Overflow

Note: The database password is the same as your hosting account password, which differs from your client area password. Stack Overflow

2. Create the `users` Table

Access **phpMyAdmin** from the control panel and execute the following SQL:

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(100) NOT NULL,
  password VARCHAR(100) NOT NULL
);

INSERT INTO users (email, password) VALUES ('test@example.com', '1234');
```

3. Develop the PHP API (`login.php`)

Create a file named `login.php` with the following content:

```
<?php
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json");

$host = "sql311.epizy.com"; // Replace with your host
$db = "epiz_12345678_flutterdb"; // Replace with your DB name
$user = "epiz_12345678"; // Replace with your username
$pass = "your_password"; // Replace with your password

$conn = new mysqli($host, $user, $pass, $db);

if ($conn->connect_error) {
    echo json_encode(["success" => false, "message" => "Connection failed"]);
    exit();
}

$email = $_POST['email'] ?? '';
$password = $_POST['password'] ?? '';

$sql = "SELECT * FROM users WHERE email='$email' AND password='$password'";
$result = $conn->query($sql);

if ($result && $result->num_rows > 0) {
    echo json_encode(["success" => true, "message" => "Login successful"]);
} else {
    echo json_encode(["success" => false, "message" => "Invalid
credentials"]);
}

$conn->close();
?>
```

Security Note: For production, use prepared statements to prevent SQL injection.

4. Upload `login.php` to InfinityFree

1. Access the **File Manager** in your InfinityFree control panel.
2. Navigate to the `htdocs` directory.
3. Upload the `login.php` file here.

You can test the API by visiting:

https://yourdomain.infinityfreeapp.com/login.php

📱 Flutter Frontend Development

1. Create a New Flutter Project

```
flutter create flutter_login_app
cd flutter_login_app
```

2. Add Dependencies

In `pubspec.yaml`, add:https://www.vrsofttech.com+1YouTube+1

```
dependencies:
  flutter:
    sdk: flutter
  http: ^0.13.5
```

Then, run:

```
flutter pub get
```

3. Implement API Service (`lib/api_service.dart`)

Create a file `api_service.dart` with the following content:

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class ApiService {
  static Future<String> login(String email, String password) async {
    var url = Uri.parse("https://yourdomain.infinityfreeapp.com/login.php");

    try {
      var response = await http.post(url, body: {
        "email": email,
        "password": password,
      });
      if (response.statusCode == 200) {
        var data = jsonDecode(response.body);
        if (data["success"]) {
          return "Login Successful!";
        } else {
          return "Login Failed: ${data["message"]}";
        }
      } else {
        return "Server Error: ${response.statusCode}";
      }
    } catch (e) {
      return "Error: $e";
    }
  }
}
```

4. Create Login Page (`lib/login_page.dart`)

Create a file `login_page.dart` with the following content:

```
import 'package:flutter/material.dart';
import 'api_service.dart';

class LoginPage extends StatefulWidget {
  @override
  _LoginPageState createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final emailController = TextEditingController();
  final passwordController = TextEditingController();
  String message = "";

  void handleLogin() async {
    String result = await ApiService.login(
      emailController.text.trim(),
      passwordController.text.trim(),
    );
    setState(() => message = result);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Flutter Login")),
      body: Padding(
        padding: EdgeInsets.all(20),
        child: Column(
          children: [
            TextField(controller: emailController, decoration:
InputDecoration(labelText: "Email")),
            TextField(controller: passwordController, decoration:
InputDecoration(labelText: "Password"), obscureText: true),
            SizedBox(height: 20),
            ElevatedButton(onPressed: handleLogin, child: Text("Login")),
            SizedBox(height: 20),
            Text(message, style: TextStyle(color: Colors.red)),
          ],
        ),
      ),
    );
  }
}
```

5. Update Main File (`lib/main.dart`)

Modify `main.dart` as follows:

```
import 'package:flutter/material.dart';
import 'login_page.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter + PHP',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: LoginPage(),
    );
  }
}
```

□ Testing the Application

1. Ensure the `login.php` API is accessible via the provided URL.
2. Run the Flutter app:[DEV Community](#)

```
flutter run
```

3. Enter the test credentials:
   - **Email**: `test@example.com`
   - **Password**: `1234`
4. Tap the "Login" button.
5. You should see a success message if the credentials are correct.

□ Important Considerations

- **InfinityFree Restrictions**: InfinityFree's free hosting is primarily designed for hosting websites and may have limitations when used for API hosting. Some users have reported issues when accessing APIs from non-browser clients, like mobile apps. [InfinityFree Forum](#)
- **Security**: Always use prepared statements in PHP to prevent SQL injection.
- **Error Handling**: Implement comprehensive error handling in both PHP and Flutter to manage unexpected scenarios gracefully.